

Quantization Summary and Sample Questions

Concept-focused summary of quantization for deep learning and generative models

Quantization can be understood as a **numerical approximation problem** rather than merely a deployment trick. The starting point is a model with full-precision parameters and activations, together with the practical desire to reduce memory footprint, bandwidth, latency, and energy. Quantization introduces a controlled loss of numerical fidelity in exchange for these deployment benefits. The central question is not simply whether low precision is possible, but **where in the model lifecycle the approximation should be handled** and **which numerical objects are most important to preserve**.

A convenient starting point is the generic uniform quantizer,

$$\hat{x} = s \cdot \text{clip}\left(\text{round}\left(\frac{x}{s}\right), q_{\min}, q_{\max}\right),$$

where s is the scale or step size, and $q_{\min}, q_{\max}$ determine the available integer codes. This formula already reveals the central tradeoff: the same scale controls both the **spacing between representable levels** and the **real-valued range** covered by those levels. If the scale is too small, the quantizer has fine resolution but severe clipping. If the scale is too large, the quantizer covers a wider range but becomes too coarse. Much of modern quantization can be read as a sequence of strategies for managing this precision-range tradeoff more intelligently.

A second foundational distinction is between **what is quantized** and **how it is quantized**. One may quantize weights, activations, or both; one may use per-tensor, per-channel, or group-wise scales; and one may keep some layers or operators at higher precision while quantizing others aggressively. This design space is already nontrivial before any training or calibration takes place. A useful way to organize it is into three phases: **pre-quantization**, **during-training quantization**, and **post-training quantization**. These are not isolated subfields, but three different times at which we may choose to handle the same underlying approximation problem.

In the **pre-quantization** phase, the central task is to understand the numerical structure of the model before low precision is imposed. Calibration is best viewed not as a mechanical preprocessing step but as a form of **distribution estimation**. One studies activation ranges, outliers, layer sensitivity, and the appropriate granularity of scales. Quantization error has at least two main sources: rounding error from a coarse grid and clipping error from an insufficient range. Pre-quantization therefore asks which tensors are hardest to compress, where outliers occur, and whether some layers should remain in higher precision. This phase is conceptually about **measurement, design, and anticipation**.

A more refined view of sensitivity comes from curvature. In one dimension, the loss increase caused by a quantization perturbation e is approximated by

$$\Delta \approx -\mathcal{L}'(w)e + \frac{1}{2}\mathcal{L}''(w)e^2.$$

Near a trained solution, the first derivative is often small, so the second derivative becomes the dominant local predictor of damage. In multiple dimensions, this becomes the Hessian-based approximation

$$\mathcal{L}(\widehat{W}) \approx \mathcal{L}(W) + \nabla \mathcal{L}(W)^\top (\widehat{W} - W) + \frac{1}{2}(\widehat{W} - W)^\top H(\widehat{W} - W).$$

The diagonal of the Hessian measures individual parameter sensitivity, while the off-diagonal terms measure coupling between parameters. This distinction helps explain why second-order post-training methods can do better than purely local rounding rules.

The **during-training** phase introduces quantization-aware training, where the quantizer is moved inside the learning loop. Instead of first learning the best floating-point model and only later compressing it, QAT learns a model that is already robust to quantization distortions. A generic objective is

$$\min_{\{W_\ell\}, \Theta} \mathcal{L}(f(x; \{Q_W(W_\ell)\}_{\ell=1}^L, \{Q_a(a_\ell)\}_{\ell=1}^L)),$$

so both weights and layer activations may be fake-quantized during training. The main difficulty here is not conceptual but differential: quantization contains rounding and clipping, both of which are hostile to ordinary gradient descent.

This is why the **straight-through estimator (STE)** matters so much. The forward pass uses the real fake-quantized values, but the backward pass substitutes a surrogate gradient, often acting like the identity inside the clipping range and zero outside it. The key idea is that QAT does not attempt to compute the exact derivative of a discrete operator; it instead uses a practical surrogate that points optimization in a useful direction. In that sense, QAT may be summarized as: **train under quantized behavior, but optimize with approximate gradients**.

Within the training-time family, two especially important learnable ideas are **PACT** and **LSQ**. PACT introduces a trainable clipping threshold α so that activations are no longer clipped by a fixed heuristic. Conceptually, this changes clipping from a manual engineering choice into a learned parameter that balances preservation of useful signal against the need for a narrower quantization range. LSQ goes one step further and learns the step size itself. In LSQ, the quantizer grid is no longer fixed beforehand; the spacing between levels becomes part of the optimization problem. A recurring theme appears here: better quantization often comes from making previously hand-chosen numerical decisions adaptive.

Binary and ternary training sit at the most aggressive end of the spectrum. Binary quantization keeps only sign information, while ternary quantization adds a zero state and therefore sparsity. These methods are useful not mainly because they dominate mainstream LLM deployment, but because they make the core approximation issue stark: as the representational alphabet becomes extremely small, the question becomes whether training can reorganize the network strongly enough to survive the compression.

The **post-training** phase starts from a pretrained model and asks how to quantize it as cheaply as possible without reopening full training. The baseline is simple **round-to-nearest (RTN)** quantization, which treats each weight independently. RTN is best viewed as the natural reference point rather than a straw man. Its value is that it cleanly exposes what more advanced PTQ methods are trying to fix: lack of data awareness, lack of layerwise reconstruction, and lack of sensitivity information.

From there the picture develops progressively richer PTQ ideas. **AdaRound** replaces naive nearest rounding by a data-driven layer reconstruction objective. Its central insight is that the best rounding direction for a given weight is not necessarily the one that minimizes scalar weight error; what matters is preservation of the **layer output** on calibration data. This shifts the viewpoint from “approximate each weight well” to “preserve the function of the layer well.”

The next step is **second-order PTQ**, illustrated through GPTQ. Here the Hessian is no longer just a mathematical curiosity; it becomes the mechanism by which the algorithm predicts which perturbations are expensive. GPTQ quantizes a column or block, computes the induced error, and then updates the remaining unquantized weights using approximate inverse-curvature information so that the downstream distortion is reduced. The key conceptual point is that first-order information answers a training question — how to descend the loss — whereas PTQ faces a different question: **if quantization forces a perturbation, how harmful will that perturbation be?** That is why curvature naturally enters.

Transformer-specific PTQ then receives dedicated treatment. **SmoothQuant** is presented as an equivalent rescaling trick for models with activation outliers. By writing

$$y = Wx = (WS^{-1})(Sx),$$

the method migrates part of the numerical difficulty from activations into weights without changing the floating-point function of the layer. This illustrates a broader principle: sometimes the best quantization strategy is not to directly quantize a troublesome tensor, but to **reparameterize the computation** so that the troublesome tensor becomes easier to quantize.

AWQ and **OmniQuant** continue this theme in different ways. AWQ is activation-aware weight-only PTQ: activations reveal which weight channels matter most, so those channels receive special protection. OmniQuant stays in the PTQ regime but makes calibration itself more learnable through **Learnable Weight Clipping (LWC)** and **Learnable Equivalent Transformations (LET)**. Conceptually, OmniQuant sits between classical fixed-parameter PTQ and full QAT: it does not retrain the network end to end, but it also does not accept fixed clipping and scaling rules as final.

The notion of deployment broadens further with **I-BERT**. Quantizing only matrix multiplications is not enough if nonlinear operators such as GELU, Softmax, and LayerNorm remain floating point. The real deployment goal is a **fully quantized computation graph**, not merely quantized weight matrices. This is one of the most important conceptual shifts in the material, because it reframes quantization as a property of the entire executed graph, not just the stored parameters.

The final part turns to **generative and multimodal models**, where the same three-phase framework remains useful but the failure modes become more subtle. In **autoregressive LLMs**, difficulty comes from activation outliers, deep residual propagation, attention sensitivity, and KV-cache accumulation during long-context decoding. In **diffusion models**, small errors can compound across denoising timesteps, and calibration must often be timestep-aware because the activation distributions evolve through the reverse process. In **diffusion transformers**, both transformer outlier issues and timestep dependence appear simultaneously. In **flow matching** and related continuous-time models, quantization perturbs a time-dependent vector field and therefore can alter the integrated trajectory of the solver itself; the literature here is still comparatively sparse.

The multimodal treatment broadens the scope further. CLIP-like dual encoders are hard to quantize because the goal is not only local task accuracy but preservation of a **shared image-text embedding geometry**. Fusion-style multimodal transformers are harder still because the visual encoder, projector, fusion blocks, and language decoder do not have equal sensitivities. The final message is that quantization is deeply modality dependent, and the hardest models are often those where low-bit errors propagate through **iterative, cross-modal, or long-context structures**.

Overall, quantization can be understood as a hierarchy of ideas. The first layer is numerical: levels, scales, range, clipping, and rounding. The second layer is structural: which tensors, layers, and operations are most sensitive. The third layer is algorithmic: calibration, QAT, PTQ, reconstruction, curvature, and equivalent transformations. The fourth layer is systems-oriented: what it means to obtain a genuinely deployable low-precision computation graph. The unifying achievement is to connect these layers into one picture: quantization is the art of deciding **when, where, and how** to absorb approximation error so that the model remains functionally useful under severe numerical constraints.

Sample Final Exam Questions

Question 1

What is the central tradeoff between scale and range, and why does a single scale create a structural tension in uniform quantization?

Sample answer

In a uniform quantizer, the scale s determines both the spacing between adjacent levels and the total real-valued range covered by the available integer codes. This means that the same parameter controls two competing objectives. If s is small, the levels are close together and rounding error is reduced, but the representable range becomes narrow and clipping error grows. If s is large, clipping is reduced because the range widens, but the grid becomes coarse and rounding error increases.

So the tension is structural, not accidental. For a fixed bit-width, one cannot improve local resolution without shrinking global coverage, or expand coverage without coarsening the grid. Much of the can be read as a search for better ways to manage this precision-range tension, either by learning scales, changing granularity, using mixed precision, or restructuring the computation itself.

Question 2

Why is the straight-through estimator conceptually necessary for quantization-aware training?

Sample answer

The problem is that the forward quantizer contains operations such as rounding and clipping, and their true derivatives are unhelpful for gradient descent. Rounding has zero derivative almost everywhere, and clipping suppresses gradients outside the allowed range. If one backpropagated through the true quantizer directly, learning would often stall or become numerically meaningless.

The STE solves this by separating forward fidelity from backward usability. The forward pass still exposes the network to quantization distortions, but the backward pass uses a surrogate derivative that allows optimization to proceed. So the STE is conceptually necessary because QAT needs the model to experience low-precision behavior while still receiving gradients that are useful enough to train on.

Question 3

Why are PACT and LSQ both examples of the broader idea that good quantization often comes from learning previously fixed numerical decisions?

Sample answer

PACT and LSQ both convert what might otherwise be hand-designed quantizer settings into trainable parameters. In PACT, the clipping threshold α is learned from the task loss rather than fixed by heuristic range selection. In LSQ, the step size itself becomes a learned quantity rather than a predetermined scale chosen only from statistics.

The broader conceptual theme is that quantization is improved when the network is allowed to adapt not only its weights but also the numerical rules that define the quantizer. This is a strong shift from early quantization schemes, where clipping and scale were often treated as engineering constants. PACT and LSQ show that these constants are actually part of the optimization problem.

Question 4

Why does GPTQ use inverse-curvature compensation after quantizing a column or block, instead of simply rounding each block independently?

Sample answer

If one rounds each block independently, the induced error is accepted as final and may accumulate through the rest of the layer. GPTQ instead recognizes that once one column is quantized, the remaining unquantized columns are still free to move. Under a second-order approximation, those remaining degrees of freedom can be adjusted in a way that absorbs part of the damage caused by the newly quantized block.

The inverse Hessian appears because it gives the locally optimal compensating update under the quadratic loss model. So GPTQ is conceptually important not just because it is a low-bit PTQ method, but because it turns quantization into a sequence of **quantize, estimate damage, compensate, continue** rather than a sequence of isolated rounding events.

Question 5

Why are diffusion models harder to quantize post-training than ordinary feedforward classifiers?

Sample answer

A feedforward classifier usually experiences quantization error only across depth: a perturbed activation is propagated through later layers in one pass. A diffusion model is harder because the denoiser is called repeatedly across many timesteps. Small quantization errors can therefore compound over the denoising trajectory, not just inside one forward pass.

A second complication is that diffusion statistics vary with timestep. This means one static calibration rule may be poor across the entire reverse process. Thus diffusion PTQ is hard for two linked reasons: the model is iterative, and its operating distribution shifts over time.